

Auditable Session Admission for Cross-Silo Federated Learning

Enzo Fenoglio
dept.Computer Science
University College London

66-72 Gower Street, London WC1E 5EA, UK
e.fenoglio@ucl.ac.uk

Philip Treleaven
dept.Computer Science
University College London

66-72 Gower Street, London WC1E 5EA, UK
p.treleaven@ucl.ac.uk

Abstract—Cross-silo federated learning keeps raw data local, but deployments frequently stall on a practical bottleneck when deciding who may invoke which session-scoped operations across organizational boundaries, under constraints that remain auditable after execution. In practice, admission is implemented via centralized policy services, platform configuration, or ad hoc checks, which drift over time and are hard to audit from boundary-visible evidence. We present a session admission gateway that enforces pre-approved session capabilities at the boundary and emits verifiable decision records. During session setup, participating organizations approve roles and constraints and mint signed session capability tokens that enumerate permitted session operations for a given *session_id*. At runtime, each request carries the token and a request-bound proof-of-possession, making replay and impersonation with stolen tokens detectable at the gateway via request binding. Admission reduces to stateless per-request cryptographic verification and capability matching at an admission gateway, while setup and orchestration remain out of scope and organization-specific. We validate the approach end-to-end on a cross-silo FL workflow using MNIST as a surrogate workload. The proof-of-concept is open-source, containerized, provisioned as code, and includes reproducible tests and evidence logs.

Index Terms—Session admission; JOSE; proof-of-possession; cross-silo federated learning; audibility.

I. INTRODUCTION

Federated learning (FL) enables multiple organizations to train a shared model without centralizing raw data. In cross-silo settings—e.g., hospitals, banks, industrial consortia—FL is often presented as a practical path to collaboration under data-locality and regulatory constraints. However, deployments can stall or become brittle when participants must decide who is allowed to invoke which session-scoped operations across organizational boundaries—particularly in cross-silo FL, where multiple organizations must pre-approve and later justify join, fetch, submit, and evaluation actions under a shared training session. These governance and administrative questions are repeatedly identified as adoption barriers and deployment-critical concerns in cross-organization FL [1], [2], especially in regulated domains such as healthcare [3], [4].

The gap between adoption and deployment is most visible at the FL session interface for creating and running a training session, joining participants, fetching the current model, submitting updates, and evaluating results. In practice,

session admission is frequently implemented through a patchwork of cloud Identity Access Management (IAM) policies, platform-specific configuration, centralized policy services, or ad hoc checks embedded in orchestration code. These are useful building blocks for authentication, infrastructure access, and internal authorization, but they do not provide portable, session-scoped admission semantics with boundary-verifiable evidence. In cross-silo deployments, they degrade for three reasons. First, they are non-portable because each organization’s identity, policies, and deployment stack differ, making integration bespoke. Second, they are hard to audit as decisions are distributed across configurations and services, making it difficult to reconstruct why an operation was allowed for a given session. Third, they are brittle under drift as operational changes can silently alter admission behavior without a durable decision record tied to the run. Our contribution targets session admission and audit evidence at the session interface boundary; privacy and robustness mechanisms for training and aggregation are orthogonal and can be layered on top.

We distinguish cross-silo session admission from in-session authorization. The difference is similar to that between a passport and a badge: the passport determines whether a participant may enter the session, whereas the badge corresponds to organization-local IAM governing what that participant may do once inside. Session admission governs the cross-boundary transition from session setup (enrollment, approvals, quorum formation) to session execution, where multiple organizations must agree on and enforce session-scoped permissions for session interface operations. Once admitted, each organization may still apply its conventional, stateful IAM (RBAC/ABAC and internal service authorization) to its own users, services, and data during session execution. Federated identity and cross-account IAM can authenticate principals and grant infrastructure-level access, but they do not define session-scoped, operation-level admission semantics for FL together with execution-time evidence that remains verifiable after the fact. Our goal is not to replace local IAM within each organization, but to supply an admission substrate at the cross-silo boundary.

This paper proposes a session admission mechanism for cross-silo FL by making each session-interface request proof-carrying. Each request includes (i) a signed *session capability*

token (SCT) encoding the permitted session operations for a specific *session_id*, and (ii) a request-bound holder-key proof. Admission is enforced at the session boundary by an admission gateway on the request path. Unlike approaches where authorization is embedded in platform configuration or orchestration logic (Section II), the proposed design enforces admission as stateless per-request verification at the session boundary, using request-carried evidence and producing explicit decision records.

Organizations approve participation and administrators mint SCTs; the roles encoded in the SCT determine the permitted operations (e.g., administrative roles for lifecycle actions such as *start_session* and *end_session*, and participant roles for session operations such as *fetch_model*, *submit_update*, and *evaluate*). Each admission decision emits a structured record (operation, *session_id*, reason code, timestamp, and hashes of the token and request) that is appended to an evidence log and correlated with run artifacts, enabling later audit without reconstructing transient platform configuration. The contribution is an FL-facing integration of standard components—multi-party authorization, signed capability tokens, and request-bound key proofs—into an admission gateway with auditable decision records.

We demonstrate the approach in a containerized cross-silo FL prototype, augmented with an admission gateway and evidence logging. The gateway enforces session-scoped permissions at the boundary and records structured evidence for both admitted and denied requests, including negative tests for capability-mismatch and possession-mismatch cases. Model artifacts are persisted in an artifact store and are accessible only under the appropriate session capabilities. The prototype reuses infrastructure from a broader federated-computing testbed [5], [6]; here we isolate and evaluate only the FL-facing session admission layer and its operational behavior. In summary, we make the following contributions:

- 1) **Session-boundary admission for cross-silo FL.** We specify an FL-native admission layer in which each session-interface request is authorized via signed session capabilities bound to a *session_id* and verified at the boundary.
- 2) **Proof-carrying requests.** We bind each session-interface request to a holder key via proof-of-possession, making stolen tokens insufficient on their own and enabling per-request admission at the session boundary.
- 3) **Auditable decision records and reproducible evidence.** We emit structured decision records and correlate them with run artifacts in a reproducible Flower-based prototype, including negative tests for capability and possession mismatch.

The remainder of the paper is organized as follows. Section 2 discusses related work. Section 3 presents the system and threat model. Section 4 specifies the session admission mechanism and verification procedure. Section 5 describes the separation of the session lifecycle into setup/orchestration and runtime admission. Section 6 reports the prototype and

experimental evidence. Section 7 discusses limitations and operational considerations, and Section 8 concludes.

II. RELATED WORK

Cross-silo FL deployments are often constrained by governance and authorization at the session boundary. Who may create/join a session, which participants may invoke which session operations, under what constraints, and with what evidence those decisions can be justified after execution. Empirical and survey-style accounts repeatedly identify governance, administrative coordination, and auditability as deployment-critical barriers in cross-organization FL. This motivates enforcing session-interface permissions through explicit boundary enforcement points and deterministic checks over request-carried evidence. This boundary-centric stance aligns with Zero Trust Architecture as standardized by NIST, which shifts security away from implicit perimeter trust toward explicit, policy-driven access decisions applied at well-defined enforcement points [7]. NIST guidance operationalizes this model for cloud-native deployments, emphasizing application-level access control and runtime enforcement [8]. Our session admission layer is consistent with this pattern when applied to cross-silo FL session interfaces. Commercial federated-computing platforms provide practical controls such as RBAC and audit logs, but admission semantics often remain platform-local [9]; we instead externalize session-interface admission as request-carried evidence checked at the boundary.

A. Complementary identity, IAM, and zero-trust mechanisms

Federated identity management (e.g., SAML and OIDC) and authorization frameworks such as OAuth are widely used building blocks for cross-organization deployments. They establish authentication, propagate identity attributes, and support delegated authorization for APIs and services [10]–[12]. Similarly, cross-account or cross-tenant IAM in cloud platforms provides practical *infrastructure-level* authorization across organizational boundaries (e.g., granting access to storage, compute, or service principals). These mechanisms are often necessary foundations for deployment, but they do not define a *multiparty session admission contract* for *session-scoped*, *operation-level* control of cross-silo FL session interfaces, nor do they yield boundary-verifiable decision evidence tied to a specific run. In practice, session admission logic is pushed into provider-specific configuration and orchestration code, where it becomes non-portable, and difficult to justify from boundary-visible evidence post hoc [1]. Our admission layer is designed to compose with these foundations by externalizing session admission semantics as signed capability artifacts verified at the session boundary and recorded as structured decision evidence (Section IV).

Zero-trust architectures provide applicable design principles — continuous verification and least privilege — but they are largely agnostic to the application-level contract of a given system. In FL, the gap is twofold: (1) an explicit mapping from session requests to session-scoped permissions and constraints, and (2) boundary-verifiable evidence of each

access decision tied to a specific run. We address both gaps with a session admission gateway that enforces session-scoped authorization at the session boundary while leaving identity and key management organization-local.

B. Capability-based authorization and proof-of-possession

Capability-based access control and signed authorization artifacts have a long history, including SPKI/SDSI-style credentials for delegated authorization [13]. More recent capability systems such as Macaroons demonstrate practical attenuation and delegation patterns suitable for distributed services [14]. Proof-of-possession mechanisms address the shortcomings of bearer tokens by binding tokens to a key held by the requester; in the OAuth ecosystem this includes DPOP [15] and related token-binding approaches. In our setting, these established mechanisms are combined into a stateless admission layer for the FL session interface.

III. SYSTEM AND THREAT MODEL

A. Setting and roles

We consider cross-silo FL among multiple organizations (e.g., Org A and Org B) that collaborate to train a shared model while keeping raw data local. Each organization operates FL clients that train on local data and interact with an FL server (or server-side service) that coordinates rounds and maintains the global model.

a) Administrative roles: Organization administrators authorize participation and mint session-scoped authorization artifacts for their organization. *Regular users* (or client processes acting on their behalf) participate in a session via a restricted set of session operations such as *fetch_model*, *submit_update*, and *evaluate*. Session lifecycle operations (e.g., *start_session*, *end_session*) are administrator-controlled.

b) Member identity and registration: Within each organization, participants are identified by an organization-scoped member identity (*sub*) and an optional holder-key binding (*cnf.jkt*) used for PoP verification. The prototype realization of member registration is described in Section VI-B.

B. Session interface

We use the term *session interface* to denote the set of session-scoped endpoints/RPCs that create and operate an FL training session across participants. We include session lifecycle actions (e.g., *start/join/end*) and participant operations such as *fetch_model*, *submit_update*, and *evaluate*. Our admission layer fronts this interface across organizations; the underlying training and aggregation procedures are unchanged. Figure 1 provides an overview of the workflow and the placement of the session admission layer that is zoomed in Figure 2; Section IV details the protocol.

C. Admission artifacts

Our mechanism uses three request-relevant elements: (i) a session identifier *session_id* binding authorization and evidence to a specific run; (ii) an SCT enumerating the session operations permitted for its holder for that *session_id*;

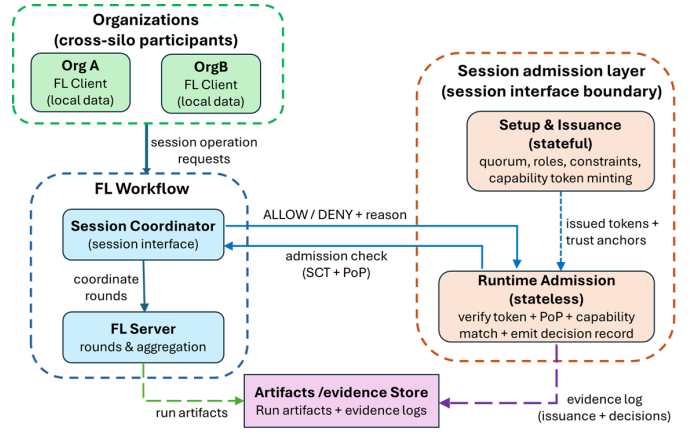


Fig. 1. Cross-silo FL workflow with a session admission layer at the session interface boundary. Setup/issuance produces organization-issued Session Capability Tokens SCTs; runtime requests are checked at the admission gateway and yield ALLOW/DENY decisions with logged evidence.

and (iii) a request-bound proof-of-possession signature demonstrating holder-key control over the request. Section IV-B specifies how SCT claims (e.g., *sub* and *cnf.jkt*) bind the token to the holder key used for PoP verification [16], [17]. Organization-specific policies and participation constraints are evaluated during session setup and compiled into the claims of the issued SCT. As a result, runtime admission depends only on the token contents and request-bound proof, without reliance on external policy state.

D. Evidence and artifact storage

Each admission decision emits a structured JSON decision record (e.g., token hash, canonical request hash, *session_id*, decision, reason, timestamps). Run artifacts (metrics and model checkpoints) are persisted to an artifact store. Model artifacts are not assumed to be publicly readable; access is mediated by session capabilities, so that possessing run evidence does not imply the right to retrieve model checkpoints.

E. Threat model and non-goals

a) Operational setting: We target cross-silo deployments in honest-but-fallible environments where heterogeneous identity/IAM stacks, configuration drift, and integration dependencies are common sources of authorization errors and audit gaps.

b) Adversary and attacks: An adversary may attempt to (i) invoke session operations without authorization, (ii) replay a stolen SCT, (iii) impersonate another principal or organization, or (iv) misuse a valid SCT by invoking an operation outside its permitted capability set or outside its bound *session_id*. Our mechanism makes each session-interface request verifiable at the boundary via token validation, request-bound PoP, and capability matching. Table I summarizes how each check addresses a specific attack vector and the evidence fields it produces.

c) *Cryptographic assumptions*: We assume standard signature primitives are secure and that organization-managed private keys are protected by a vault; key compromise, revocation, and recovery are treated as limitations (Section VII).

d) *Non-goals*: We do not address privacy or robustness mechanisms for the FL learning procedure itself (e.g., secure aggregation, differential privacy, or defenses against malicious model updates). These mechanisms are complementary and operate within the learning procedure after requests have been admitted, whereas our focus is the deployability and auditability of cross-silo authorization for the session interface.

e) *Failure semantics*: A DENY decision blocks only the requested operation; it does not terminate the session. A session may be created once quorum is reached and remain inactive until permitted operations occur (Section V-B).

IV. SESSION ADMISSION MECHANISM

A. Design goals

We aim to make cross-silo authorization for the *session interface* (i) *portable* across heterogeneous organizational stacks, (ii) *verifiable at the boundary* from request-carried evidence, and (iii) *auditable after execution* via run-tied decision records. The mechanism is designed to compose with federated identity and cloud IAM rather than replace them. Identity and infrastructure access may remain organization and provider-specific, while session-scoped authorization is enforced uniformly at the admission gateway. Figure 2 illustrates the split between stateful setup and per-request runtime admission. Conceptually, the chain separates trust-anchor configuration, issuance of session-scoped permissions, and per-request holder-key binding, enabling deterministic cryptographic verification at the session boundary.

B. Session capability tokens

A *session capability token (SCT)* is a signed artifact minted during session setup by an organization administrator. We assume administrators authenticate using mutual TLS (mTLS) over TLS [18]; OAuth mutual-TLS profiles provide one concrete realization when OAuth is used [16]. An SCT binds permissions to a specific session and to a holder identity under the issuing organization trust domain. Concretely, the token contains at least:

- `session_id`: session identifier;
- `issuer`: organization identifier (e.g., Org A);
- `sub`: holder identifier within the issuing organization domain;
- `cnf.jkt`: holder-key binding (thumbprint of the holder public key) used for PoP verification;
- `capabilities`: a set of permitted session operations (e.g., `{fetch_model, submit_update, evaluate}`);
- `constraints`: optional predicates (time window, round bounds, rate limits, etc.);
- `exp`: expiration/validity interval.

We use `sub` to identify the holder within the issuing organization’s namespace, and `cnf.jkt` [17], [19] to bind the token to the holder’s PoP verification key by carrying the

thumbprint of the holder public key. The SCT is signed by an organization-managed key (stored in a secure vault), and the admission gateway is configured with the corresponding trust anchors/public keys for participating organizations.

a) *Realization with standard token formats*: SCTs can be realized using the JOSE (JSON Object Signing and Encryption) framework: a signed JSON Web Token (JWT) carrying the session claims and capability set, signed using the JSON Web Signature (JWS), with issuer public keys distributed as JSON Web Keys (JWKs) [20]–[22]. If deployments require explicit holder-of-key semantics, standardized confirmation claims can associate the token with a proof key [17]. When stable key identifiers are needed for verification material selection, JWK thumbprints provide a standard, collision-resistant identifier [19]. JWT best-current-practice guidance applies to avoid common validation pitfalls [23]. We remain format-agnostic at the abstraction level; these references illustrate a widely deployed realization.

C. Request-bound proof-of-possession

Bearer tokens alone are vulnerable to replay and impersonation when stolen. Each request includes a *proof-of-possession* (PoP) signature under the holder’s private key over a canonical representation of the request, so a stolen SCT alone cannot be used to invoke session operations. We adopt the DPop approach from RFC 9449 [15], signing a JWT that includes:

- HTTP method and endpoint (or RPC method name),
- `session_id`,
- a nonce or unique request identifier (`jti`),
- a timestamp (and optional TTL),
- a hash of the request body where applicable.

The admission gateway verifies that the PoP signature corresponds to the holder identity asserted by the SCT (via `cnf.jkt`) and that the signed claims bind to the request being evaluated. This prevents token replay even if an SCT is intercepted, as the attacker cannot forge valid request-specific signatures without the holder’s private key.

a) *Canonical hashing*: To ensure that request hashes and evidence fields are stable across implementations, JSON inputs are canonicalized before hashing. We apply the JSON Canonicalization Scheme (JCS, RFC 8785 [24]) to request bodies and decision records, providing deterministic request/evidence hashing regardless of key ordering, whitespace, or number formatting.

D. Admission gateway procedure

The admission gateway fronts the session interface and returns ALLOW or DENY for each request. In our architecture, enforcement is canonical at the coordinator boundary (Figure 2). The coordinator invokes the admission check for session-scoped operations it forwards to the session interface; clients may optionally perform local sanity checks (e.g., token expiry and operation presence) for usability without being authoritative. Verification consults only request-carried evidence and configured trust anchors; a bounded replay cache (optional) is an implementation detail. Verification consists of:

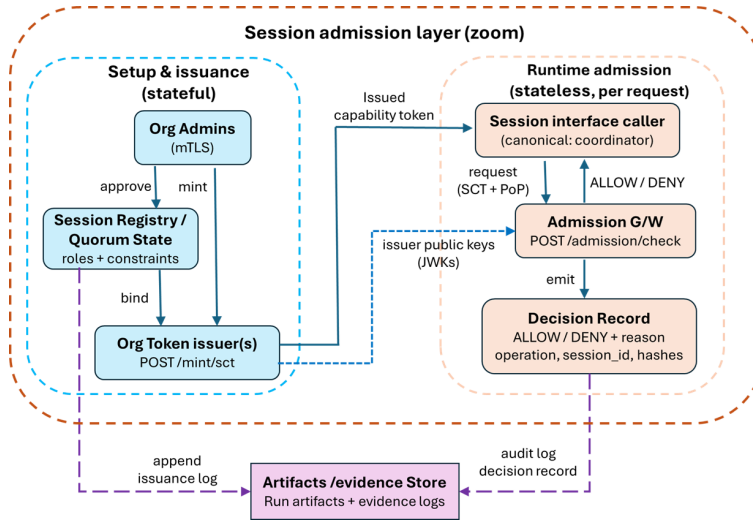


Fig. 2. Session admission layer (Zoom). During setup, administrators approve quorum roles/constraints and mint SCTs. At runtime, the coordinator submits session requests carrying the SCT and PoP; the admission gateway returns ALLOW/DENY, emits a decision record (*operation*, *session_id*, hashes), and appends evidence to the artifact/evidence store.

- 1) **SCT validation:** verify signature, issuer trust anchor, expiration, and required fields; parse *session_id*, capability set, and any constraints.
- 2) **Capability match:** map the requested endpoint/RPC to an abstract session operation and check that it is permitted by the SCT for the bound *session_id*; evaluate any token constraints.
- 3) **PoP validation:** reconstruct the canonical request string and verify the PoP signature; check nonce/timestamp freshness to reduce replay.
- 4) **Decision record:** emit a structured record (Section III) with ALLOW/DENY and a reason code.

A denied request blocks only the requested operation. The session may continue for other participants and operations that satisfy their capabilities.

V. SESSION LIFECYCLE: SETUP VS. RUNTIME ADMISSION

A. Separation of stateful setup from per-request admission

A core design choice is the separation between:

- **Session setup and orchestration (stateful):** enrollment workflows, approval/quorum formation, and minting of organization-issued SCTs;
- **Runtime admission (stateless per request):** boundary checks at the admission gateway using request-carried evidence (SCT + PoP) and configured trust anchors.

This split confines mutable coordination state (quorum, roles, constraints) to setup, but keeps runtime authorization at the session boundary (Section IV).

B. Quorum and session creation

We assume a session is created once a required quorum of organizations has been reached via an administrative workflow. Session creation establishes *session_id* and the approved roles/constraints; administrators then mint SCTs bound to

that identifier for the permitted operations. A newly created session may be *inactive*, i.e., no training rounds execute until permitted participants invoke session operations consistent with their minted capabilities. Once quorum is reached and setup artifacts are available, the orchestrator initiates training. The admission gateway regulates access to session-scoped operations throughout execution by evaluating request-carried evidence.

C. Role-scoped operations

We separate administrative lifecycle operations from participant operations to reduce privilege scope and simplify review. In the prototype described in Section VI, administrators mint SCTs and manage lifecycle actions. Regular users are limited to participant operations (e.g., *fetch_model*, *submit_update*, *evaluate*). Server-side aggregation is not exposed as a regular-user operation.

D. Operational mapping to session interface endpoints

Session interface endpoints (or RPC methods) are mapped to abstract operations used for capability matching at the gateway. This mapping is explicit and versioned with the gateway configuration to avoid implicit coupling to orchestration code. For example, */session/model* maps to *fetch_model*, */session/update* maps to *submit_update*, and */session/evaluate* maps to *evaluate*. Administrative endpoints map to lifecycle operations (e.g., *start_session*, *end_session*). The SCT capability set is evaluated against these abstract operations rather than raw URLs to keep the policy stable under API refactors.

E. Decision evidence as a run artifact

Each admission decision appends a structured decision record to the evidence log, including *session_id*, operation, decision (ALLOW/DENY), a reason code, and hashes of

TABLE I
SESSION ADMISSION CHECKS, ADDRESSED FAILURE MODES, AND MINIMAL EVIDENCE FIELDS FOR AUDIT JUSTIFICATION.

Admission check	Prevents/detects	Evidence fields
Token signature & validity PoP verify PoP freshness (nonce/time) Capability match (operation) Session binding	Forged/expired tokens Impersonation with stolen token Replay of previously valid requests Unauthorized session operation Cross-session token misuse	token hash, iss, exp, signature verification outcome, decision request hash, PoP verification outcome jti, timestamp, freshness outcome operation, capability set id/hash, reason code session_id, token session_id claim

the SCT and the request (under the canonicalization used for PoP). This log can be correlated with run artifacts (model checkpoints and metrics) for later inspection without reconstructing transient IAM configuration or orchestration state.

VI. PROTOTYPE AND EVIDENCE

A. Implementation overview

We evaluate the session admission layer in a containerized cross-silo FL deployment built on a standard FL framework and a canonical image-classification workload. The deployment includes (i) an FL server and per-organization FL clients, (ii) a coordinator/orchestrator that manages session progression, (iii) an admission gateway fronting the session interface and performing stateless verification (Section IV), and (iv) evidence and artifact stores for decision records and run outputs.

The proof-of-concept is packaged as containers and provisioned as code with OpenTofu [25] to support reproducible deployment across organizations. The FL framework is Flower [26], and the workload is MNIST [27], used as a controlled surrogate to exercise session operations, admission decisions, and evidence generation end-to-end. We model organizational cohorts by partitioning digits across participants (e.g., even/odd or subsets), treating `digit` as a stand-in for a governed data category (analogous to a clinical cohort label) so that capability rules can permit or deny specific evaluation/prediction requests. We release a reproducible proof-of-concept implementation and test on GitHub¹. Evaluation on domain workloads and heterogeneous cross-organizational datasets is left to future works.

Organization keys for SCT signing and member PoP are managed in organization-controlled vaults. Our prototype simulates vault storage on disk; production deployments would use hardware-backed key management services (e.g., AWS Nitro Enclaves, Azure Key Vault, or HSM-backed vaults). The admission gateway is configured with the corresponding public keys and trust anchors for participating organizations. This matches common operational practice in cross-silo deployments, where each party manages its own signing material and shares only the verification material required for cross-organization admission.

B. Experimental setup and session interface

Our objective is to validate that session admission is (i) enforceable at the session boundary, (ii) portable across or-

ganizations without shared runtime policy services, and (iii) auditable after execution from boundary-visible evidence. We therefore focus on session-scoped operations exposed by the session interface, rather than on learning performance.

We instantiate the two roles from Section III-A in the prototype. Administrators control session lifecycle operations and mint SCTs during setup. Regular users are restricted to participant operations (*fetch_model*, *submit_update*, *evaluate*); server-side aggregation is not exposed as a regular-user operation.

a) *Member registration (prototype prerequisite)*: Before SCT issuance, each organization registers members by recording `{org_id, member_id, public key, jkt}` in an organization-local registry, where `member_id` serves as the member’s identity within that organization’s namespace and `jkt` is the thumbprint of the member’s public key. SCTs reference the registered member via `sub` (which equals `member_id` in the issuing organization’s registry) and carry the corresponding holder-key binding as `cnf.jkt`; PoP proofs are generated using the corresponding private key held by the member (e.g., in an organization-managed vault). This avoids relying on free-form usernames in the UI while preserving local key custody.

C. Test protocol

We test whether the admission gateway enforces session-scoped permissions and binds each request to the holder key via PoP. The protocol is structured as a small suite of end-to-end tests that exercise both success and failure modes commonly observed in cross-silo deployments.

Session setup and issuance: Administrators approve participation constraints and mint SCTs for a shared `session_id` (Section III-C). We issue participant-role and administrative-role tokens to exercise participant operations versus lifecycle operations in the tests. All SCTs are signed with organization-managed keys.

- 1) **Positive path**. Clients invoke *fetch_model* and *submit_update* with a valid SCT and a request-bound PoP. The gateway returns `ALLOW` and forwards the request to the session interface. Training proceeds and produces standard run metrics and model artifacts.
- 2) **Negative case 1 (capability mismatch)**. A client attempts an operation that is not included in its capability set, such as calling *evaluate* using an SCT that omits *evaluate*. The gateway returns `DENY` with a capability-mismatch reason code.

¹<https://github.com/onzeif/FLICS-cross-silo-admission>

- 3) **Negative case 2 (PoP mismatch / replay).** A request reuses an SCT with an invalid or stale PoP (e.g., wrong holder key, incorrect binding to the canonical request, or failed freshness check). The gateway returns DENY with a mismatch or freshness failure reason, aligning with standardized mechanisms designed to detect replay of stolen bearer material [15].

These tests deliberately mirror operational failure modes that are difficult to diagnose when authorization is embedded in configuration or orchestration logic. The goal is not cryptographic novelty, but to demonstrate that a session admission layer makes such failures explicit, localizable, and auditable at the session boundary.

D. Findings and evidence

The primary result is that session admission can be enforced at the session boundary and reconstructed after execution from decision records. The evaluation focuses on validating the correctness and audibility of stateless admission at the session boundary, rather than on domain realism or large-scale performance. Requests are admitted only when SCT validation, PoP verification, and capability matching succeed. The gateway denies requests that fail these checks before they reach the session interface, returning reason codes for capability mismatches or PoP failures. For each request, the evidence log records a JSON decision including `session_id`, operation name, canonical request hash, SCT hash (optionally with issuer/subject metadata), and outcomes for PoP verification and freshness checks. Each record also includes the decision, reason code, timestamps, and a correlation identifier linking to run artifacts. Run artifacts comprise a `run.json` record (session and workload configuration), training metrics, and model checkpoints. Checkpoint retrieval is treated as a capability-checked session operation. Table I summarizes the checks, failure modes, and minimal evidence fields used for post hoc admission justification.

E. Overhead and deployability considerations

The admission gateway performs per-request cryptographic verification and capability matching, which is independent of the number of session participants (per-request verification and lookup) and does not depend on the learning algorithm. Because admission applies to session-scoped operations, it primarily affects the frequency and latency of session-interface RPCs rather than the computational cost of local training. In practice, public keys and token parsing can be cached across requests within a session, and the gateway can be horizontally scaled as a stateless service. These properties are aligned with the deployment goal of making authorization decisions explicit at the boundary without introducing a centralized policy evaluation dependency on the critical request path.

Microbenchmark (admission overhead): Table II reports a microbenchmark of the admission gateway (G/W) to provide a sanity check on the per-request cost of session admission, where ALLOW is a valid SCT+PoP, capability permitted (Positive path), DENY-Cap is a valid SCT+PoP but with capability

TABLE II
ADMISSION G/W PERFORMANCE (MEDIAN OVER N=1000 REQUESTS)

Operation	ALLOW [ms]	DENY-Cap [ms]	DENY-PoP [ms]
Token verify	0.126	0.125	0.124
PoP verify	0.094	0.095	0.124
Capability match	0.003	0.003	N/A
Full admission check	0.261	0.261	0.255

violation (Negative case 1), and DENY-PoP is a SCT valid but with PoP binding mismatch and capability match not reached (Negative case 2). The benchmark was run on a single host (Intel Xeon W5-3425, 24 cores, 3.20GHz base / 4.60GHz max turbo, 30MiB L3; 128GiB DDR5-4800; Ubuntu 22.04 LTS, kernel 6.8.0-94-generic HWE; Docker 27.5.1; performance governor, no concurrent load). The gateway container ran with no CPU or memory caps. Cryptographic operations use Ed25519 via libsodium 1.0.18.

Methodology: Each iteration uses a fixed, valid SCT for a fixed `session_id` and generates a fresh request-bound PoP (new `jti/timestamp/nonce`) over a canonical request representation. The measured path therefore covers the full runtime admission procedure: (i) token signature and claim validation; (ii) PoP signature and claim validation (`htu, htm, freshness`); (iii) capability matching for the requested operation; and (iv) decision-record construction. Timings are taken inside the gateway using a monotonic high-resolution clock. To avoid disk I/O and logging dominating sub-millisecond measurements, per-iteration timing records are appended to an in-memory buffer and flushed once at the end of the run; the reported numbers thus reflect admission computation rather than artifact-store persistence costs.

Results: End-to-end admission is sub-millisecond and dominated by cryptographic verification (SCT signature validation and PoP verification). Capability matching is an in-memory lookup and contributes negligibly. The PoP-mismatch path short-circuits before capability evaluation (reported as N/A), but total latency remains similar because signature verification dominates. These measurements indicate that per-request admission is unlikely to materially affect the critical path of FL training and aggregation. Moreover, the reported numbers reflect single-request latency under no concurrent load, while throughput and tail latency under parallel sessions are out of scope.

F. Reproducibility

The prototype is packaged as containers and produces two classes of artifacts that support reproducibility. First, the evidence log provides request-level traces of admission decisions with reason codes and hashed bindings to canonical requests. Second, the artifact store provides run-level metadata (`run.json`) and standard FL metrics, as well as persisted model checkpoints protected by session-scoped access. Together, these artifacts enable re-checking admission decisions post hoc, correlating them with the corresponding

run, and separating authorization failures from learning-related variability.

VII. LIMITATIONS AND OPERATIONAL CONSIDERATIONS

a) Horizontal FL and Session-Bound Admission: Our prototype and evaluation target cross-silo, *horizontal FL* settings where organizations already hold local training datasets. Admission governs *session-scoped operations* over these existing data holders. We do not address *vertical FL* (feature-partitioned collaboration) [28] nor federated data-acquisition lifecycles covering ingestion, consent, provenance, and retention beyond session operations. Adapting the session interface and capability model to vertical FL, and extending admission beyond the session interface, are out of scope in this work.

b) Key compromise and revocation: We assume organization-managed key custody (e.g., vault-backed) and do not treat key compromise, revocation, or recovery. Short-lived SCTs, session termination, and key rotation mitigate key-loss exposure; mid-session revocation semantics are not treated.

c) Evidence integrity model: Decision records are stored as structured JSON with hashes to support later audit from boundary-visible artifacts (e.g., SCT, PoP). Stronger integrity mechanisms (e.g., append-only tamper-evident logs) can be substituted without changing admission semantics.

d) Evaluation scope: Our evaluation validates the admission architecture: correct boundary enforcement, explicit failure handling, and evidence generation. It is not intended as a production-deployment study or a large-scale performance evaluation. Admission overhead is driven by per-request cryptographic verification (Table II); practical optimizations include caching verification material and horizontally replicating the stateless admission gateway.

e) Complementary FL security mechanisms: We do not address privacy and robustness mechanisms for training and aggregation (Section III-E.d).

f) Operational considerations: Deployments must address clock skew tolerance for freshness checks and secure trust-anchor distribution. These are deployment-specific engineering constraints that we treat as operational prerequisites rather than protocol requirements.

VIII. CONCLUSION

Cross-silo federated learning deployments are often hindered by a practical gap in portable, and auditable authorization for session-scoped operations across organizational boundaries. We introduce a session admission layer at the session interface that uses signed SCTs and request-bound proof-of-possession PoP, enabling per-request boundary verification and structured decision records suitable for audit. Our prototype demonstrates end-to-end enforcement and evidence generation, including capability-mismatch and PoP-mismatch denial cases. The admission layer complements existing identity and IAM foundations while leaving training and aggregation procedures unchanged. Future work includes operational hardening (e.g., key rotation and revocation workflows) and broader evaluation across additional FL stacks and deployment configurations.

REFERENCES

- [1] T. Barbereau, J. Delgado Fernandez, and S. Potenciano Menci, "The governance of federated learning: a decision framework for organisational archetypes," *Data & Policy*, vol. 7, p. e53, 2025.
- [2] K. Sprenkamp, J. D. Fernández, S. Eckhardt, and L. Zavolokina, "Overcoming intergovernmental data sharing challenges with federated learning," *Data & Policy*, vol. 6, p. e27, 2024.
- [3] J. Kommusaar *et al.*, "A roadmap for federated learning projects using health data to guide sustainable Artificial Intelligence development in the European Union," *International Journal of Medical Informatics*, vol. 208, p. 106242, 2026. Epub 2025-12-25.
- [4] J. Xu, B. S. Glicksberg, C. Su, P. Walker, J. Bian, and F. Wang, "Federated learning for healthcare informatics," *Journal of Healthcare Informatics Research*, vol. 5, pp. 1–19, 2021.
- [5] E. Fenoglio and P. Treleaven, "Federated Computing as Code (FCaC) Sovereignty-aware Systems by Design," *arXiv preprint arxiv:2603.17331*, 2026.
- [6] E. Fenoglio and P. Treleaven, "Federated computing: information integration under sovereignty constraints," *Royal Society Open Science*, vol. 13, p. 251318, 02 2026.
- [7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Gaithersburg, MD, Aug. 2020.
- [8] R. Chandramouli and Z. Butcher, "A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments," NIST Special Publication 800-207A, National Institute of Standards and Technology, Gaithersburg, MD, Sept. 2023.
- [9] Rhino Health, "Rhino federated computing platform (rhino fcp) — platform," <https://www.rhinofcp.com/platform>. Accessed 2026-02-22.
- [10] S. Cantor, J. Kemp, R. Philpott, and E. Maler, "Assertions and Protocols for the OASIS Security Assertion Markup Language (SAML) V2.0," OASIS Standard, 2005.
- [11] OpenID Foundation, "OpenID Connect Core 1.0 incorporating errata set 2," 2014.
- [12] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, IETF, 2012.
- [13] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, "SPKI Certificate Theory," RFC 2693, IETF, 1999.
- [14] A. Birgisson, M. Vasan, N. Vijaykumar, V. Ernstsson, *et al.*, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud," in *Network and Distributed System Security Symposium*, 2014.
- [15] D. Fett, M. Jones, J. Bradley, A. Gropper, H. Tschofenig, B. Campbell, and J. Panzer, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)," RFC 9449, IETF, 2023.
- [16] B. Campbell, J. Bradley, N. Sakimura, and T. Matsumoto, "OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens," RFC 8705, IETF, 2020.
- [17] M. Jones and B. Campbell, "Proof-of-Possession Key Semantics for JSON Web Tokens (JWTs)," RFC 7800, IETF, 2016.
- [18] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, 2018.
- [19] M. Jones, "JSON Web Key (JWK) Thumbprint," RFC 7638, IETF, 2015.
- [20] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
- [21] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Signature (JWS)," RFC 7515, IETF, 2015.
- [22] M. Jones, "JSON Web Key (JWK)," RFC 7517, IETF, 2015.
- [23] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token Best Current Practices," RFC 8725, IETF, 2020.
- [24] A. Rundgren, "JSON Canonicalization Scheme (JCS)," RFC 8785, IETF, 2020.
- [25] OpenTofu Project, "OpenTofu Documentation," <https://opentofu.org/docs/>, 2026. Accessed 2026-02-10.
- [26] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. d. Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.
- [27] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits," 1998. Accessed: 2026-01-18.
- [28] Y. Liu, Y. Kang, T. Zou, Y. Pu, Y. He, X. Ye, Y. Ouyang, Y.-Q. Zhang, and Q. Yang, "Vertical federated learning: Concepts, advances, and challenges," *IEEE Trans. on Knowl. and Data Eng.*, vol. 36, p. 3615–3634, July 2024.